Expense Manager App — Project Requirements Document

Oct 28, 2026 · @KnowHow Dev 001

Expense Manager is an offline-first mobile app (Android and iOS, built in Flutter) for logging expenses and income by category and reviewing them by Today, Week, Month or Year, with budgets and charts. Everything is stored on the device; no account or internet connection is needed in v1.

1. Overview

Purpose: give one person a fast, private way to see where their money goes and whether they are staying inside budget.

Goals

- Record an expense or income in under 10 seconds from the home screen.
- Filter every total and list by Today, Week, Month, Year or a custom date range.
- Show income vs expenses and spending by category as clear graphs.
- Warn the user before a monthly or category budget runs out.
- Work fully offline, with data kept on the phone and exportable.

Success measures (v1)

- All features in section 4 marked *Must* pass the acceptance criteria in section 14.
- App opens to the Home screen in under 2 seconds on a mid-range Android phone.
- Totals on Home, Records and Reports always match for the same period.

Design reference: the clickable prototype `expense-manager-prototype.html` (7 screens + build spec JSON) and the Design canvas "Expense Manager App".

2. Scope

Version 1 is a single-user, on-device app; cloud sync and shared budgets come later.

In scope (v1)	Out of scope (v1)
Add, edit, delete expenses and income	User accounts, login, cloud sync
Default + custom categories (add, edit, reorder, delete)	Bank / SMS auto-import of transactions

In scope (v1)	Out of scope (v1)
Period filters: Today, Week, Month, Year, custom range	Multi-currency conversion
Records list with search and type filter	Shared or family budgets
Monthly overall and per-category budgets with alerts	Investment or loan tracking
Reports: bar, donut and line charts	Web or desktop release
Wallets (Cash, Bank, Card, e-Wallet)	Receipt OCR (reading amounts from photos)
Monthly recurring records	Ads or in-app purchases
Receipt photo attachment	
App lock (PIN / fingerprint), daily reminder	
Dark mode, currency, week start, language settings	
Export CSV / PDF, local backup and restore	

3. Users and use cases

The primary user is an individual managing personal or household money in Sri Lankan rupees, using an Android phone first.

1. **Log an expense** — tap +, enter amount, pick category, save. Date defaults to now, wallet to the last one used.
2. **Log income** — same flow with the Income toggle (salary, freelance payment, gift).
3. **Check today or this week** — open Home, switch period, read balance, income, expenses and recent records.
4. **Find a record** — Records screen, search by note, category or amount, filter by period and type.
5. **Fix a mistake** — open a record, edit or delete it; totals update everywhere immediately.
6. **Stay inside budget** — set a monthly budget and category limits; get a warning at 80% and see overspending.
7. **Understand spending** — Reports screen: income vs expenses over time, spending by category, this month vs last month.
8. **Automate fixed items** — mark rent, bills or salary as repeating monthly.
9. **Keep data safe** — lock the app, export to CSV/PDF, back up and restore.

4. Functional requirements

48 requirements across 7 modules; *Must* items make the v1 release, *Should* items follow in the same version if time allows, *Could* items move to v1.1 if needed.

ID	Module	Requirement	Priority
FR-01	Home	Period switch Today / Week / Month / Year; default Month; last choice remembered	Must
FR-02	Home	Period label with previous / next arrows to step through periods	Must
FR-03	Home	Balance card: net balance, total income, total expenses for the selected period	Must
FR-04	Home	Budget card: spent vs budget, % used, amount and days left; tap opens Budgets	Should
FR-05	Home	Five most recent records in the period; See all opens Records	Must
FR-06	Add / edit	Expense / Income toggle that switches accent colour and category set	Must
FR-07	Add / edit	Amount field with decimal keyboard; required, greater than 0, max 2 decimals	Must
FR-08	Add / edit	Category grid (required); last-used category preselected; Edit link to Categories	Must
FR-09	Add / edit	Date and time picker, default now	Must
FR-10	Add / edit	Paid with: Cash, Card, Bank, e-Wallet (from the user's wallets)	Should
FR-11	Add / edit	Optional note, max 200 characters	Must
FR-12	Add / edit	Attach a receipt photo from camera or gallery	Could
FR-13	Add / edit	Repeat every month switch that creates a recurring rule	Should
FR-14	Add / edit	Save validates and stores the record, returns, shows a snackbar with Undo	Must
FR-15	Add / edit	Tap any record to edit it; delete asks for confirmation	Must

ID	Module	Requirement	Priority
FR-16	Records	Period switch Today / Week / Month / Year plus custom date range	Must
FR-17	Records	Type filter All / Expenses / Income	Must
FR-18	Records	Search by note, category name or amount	Should
FR-19	Records	Income and expense totals for the current filter	Must
FR-20	Records	List grouped by day (Today, Yesterday, Mon 26 Oct) with daily net, newest first	Must
FR-21	Records	Empty state when no records match	Must
FR-22	Reports	Period switch Week / Month / Year	Must
FR-23	Reports	Summary: income, expenses, saved % or overspent amount	Must
FR-24	Reports	Grouped bar chart of income vs expenses (section 9)	Must
FR-25	Reports	Donut chart of spending by category with legend and %	Must
FR-26	Reports	Category breakdown list; tap opens Records filtered to that category	Should
FR-27	Reports	Spending pace line chart, this month vs last month (Month only)	Could
FR-28	Reports	Export the report as PDF	Could
FR-29	Budgets	Overall monthly budget ring: % used, spent of limit, safe to spend per day	Should
FR-30	Budgets	Per-category monthly budgets with states: normal, 80% warning, fully used, over	Should
FR-31	Budgets	Create, edit and delete budgets	Should
FR-32	Budgets	Notification when a budget reaches 80% (on/off)	Could
FR-33	Categories	Expense / Income tabs with records-this-month count per category	Must
FR-34	Categories	Add or edit a category: name, icon, colour	Must

ID	Module	Requirement	Priority
FR-35	Categories	Drag to reorder; the order is used in the Add grid	Should
FR-36	Categories	Delete a category; if it has records, ask which category to move them to	Must
FR-37	Categories	Seed the default categories on first launch (section 6)	Must
FR-38	Settings	Currency, default LKR (Rs)	Must
FR-39	Settings	Week starts on, default Monday	Must
FR-40	Settings	Theme: System / Light / Dark	Should
FR-41	Settings	Language: English first; Sinhala and Tamil later	Could
FR-42	Settings	App lock with PIN or fingerprint	Should
FR-43	Settings	Daily reminder at a chosen time, default 8:00 PM	Should
FR-44	Settings	Manage wallets and accounts	Should
FR-45	Settings	Recurring records list: pause, edit, delete	Should
FR-46	Settings	Export records to CSV and PDF for a chosen range via the share sheet	Should
FR-47	Settings	Back up all data to one file and restore from it	Should
FR-48	Settings	Profile name shown on the More screen	Could

5. Screens, navigation and design

Seven screens, reached through a bottom bar of four tabs plus a centre + button; layouts follow the prototype at 390 × 844 logical px.

Screen	Reached from	Main contents
Home	Bottom bar: Home (start screen)	Period switch, period arrows, balance card, budget card, recent records
Records	Bottom bar: Records; Home See all / search	Search, period switch, Custom range, All / Expenses / Income, totals, day-grouped list

Screen	Reached from	Main contents
Add / edit record	Centre + button; tap a record	Type toggle, amount, category grid, date, paid with, note, receipt, repeat, Save
Reports	Bottom bar: Reports	Week / Month / Year, summary tiles, bar chart, donut + breakdown, pace line chart, Export
Budgets	Home budget card; More › Budgets	Overall ring, per-category budgets, New budget, 80% alert switch
Categories	More › Categories; Add screen Edit link	Expense / Income tabs, list with edit and reorder, Add category
More (settings)	Bottom bar: More	Profile, money settings, preferences, security and reminders, data

Bottom bar: 84 px tall — Home, Records, + (56 px square, radius 18, ink fill, raised 24 px), Reports, More. Active tab in ink, others muted.

Design tokens

Token	Value	Used for
Background	#F2F4F7	Screen background
Surface	#FFFFFF	Cards, lists, inputs
Ink	#0F1B2D	Text, primary buttons, balance card, selected chips
Muted text	#5A6575	Labels, secondary lines (5.9:1 on white)
Segment track	#E3E7ED	Period / type switch background
Divider	#EEF1F4	List row separators
Border	#D5DBE3	Chip and button outlines
Income	#1D5FCC	Income amounts, income toggle, progress bars
Expense	#B4471A	Expense amounts, expense toggle, over-budget state
Expense chart bar	#F08A4B	Expense bars in charts

Token	Value	Used for
Warning	#E07B39	Budgets at 80% or more
On-dark income / expense	#9EC2FF / #FFB48C	Labels on the ink balance card

- **Fonts:** Sora 500–700 for titles and numbers; Manrope 400–800 for body text. Bundle both in the app so they work offline.
- **Type sizes:** screen title 26, section title 15–17, row title 15, secondary 12–13, balance 34.
- **Shape:** cards radius 20, balance card 24, buttons 14, chips fully round, category icon tile radius = 32% of its size.
- **Spacing:** screen padding 20, section gap 16, list row 10 vertical; minimum touch target 44 × 44.
- **Dark mode:** same layout; background #0B1220, surface #141D2B, text #E8ECF2; keep income/expense hues, lighten them to pass 4.5:1 contrast.
- **Accessibility:** labels on icon-only buttons, colour never the only signal (amounts also carry + / −), text contrast at least 4.5:1.

6. Default categories

17 categories are seeded on first launch (10 expense, 7 income); users can rename, recolour, reorder or delete any of them. Icons are 24 × 24 stroke icons on a tinted tile.

Type	ID	Name	Icon	Tile colour	Icon colour
Expense	food	Food & dining	utensils	#FDE9DC	#B4471A
Expense	groceries	Groceries	shopping cart	#E4F4DD	#3C7A1E
Expense	transport	Transport	car	#E1ECFB	#1D5FCC
Expense	bills	Bills & utilities	receipt	#FFF2CC	#8A6100
Expense	home	Rent & housing	house	#E6EAF0	#3B4A5E
Expense	shopping	Shopping	shopping bag	#F6E3F1	#9B2F7E
Expense	health	Health	heart pulse	#FBE2E4	#B42335
Expense	fun	Entertainment	play circle	#ECE6FB	#5B3CC4
Expense	education	Education	graduation cap	#DFF3F0	#0F7468
Expense	other	Other	three dots	#ECEEF1	#4A5361
Income	salary	Salary	briefcase	#DCEBFF	#1652B8
Income	freelance	Freelance	laptop	#DDF1F4	#0E6E7E
Income	business	Business	store	#E9E4FA	#4D35B0

Type	ID	Name	Icon	Tile colour	Icon colour
Income	invest	Investments	trending up	#DDF3E6	#1C7A44
Income	gift	Gifts	gift	#FCE4EC	#B0245C
Income	refund	Refunds	rotate back	#E6EAF0	#3B4A5E
Income	other_income	Other income	three dots	#ECEEF1	#4A5361

The *Other* categories cannot be deleted, so every record always has a valid category.

7. Data model

Five SQLite tables (via drift) plus key–value settings; amounts are stored as integer cents to avoid rounding errors.

transactions

Field	Type	Rules
id	int, PK	Auto increment
type	text	`expense` or `income`
amount_cents	int	> 0; Rs 2,250.50 = 225050
category_id	int, FK → categories	Required; category type must match record type
wallet_id	int, FK → wallets	Required
occurred_at	datetime	Local date and time of the record; indexed
note	text, nullable	Max 200 characters
receipt_path	text, nullable	File path in app storage
recurring_rule_id	int, FK, nullable	Set when created by a recurring rule
created_at / updated_at	datetime	Set automatically

categories

Field	Type	Rules
id	int, PK	

Field	Type	Rules
type	text	`expense` or `income`
name	text	Required, unique per type, max 30 characters
icon	text	Icon key, e.g. `utensils`
color_bg / color_fg	text	Hex colours
sort_order	int	Order in lists and the Add grid
is_default / is_locked	bool	Locked = the Other categories

Other tables

Table	Fields
wallets	id, name, icon, sort_order, is_archived
budgets	id, category_id (null = overall budget), period (`month`), limit_cents, created_at
recurring_rules	id, type, amount_cents, category_id, wallet_id, note, frequency (`monthly`), day_of_month, next_run_at, is_active
settings (key–value)	currency_code (LKR), currency_symbol (Rs), week_start (monday), language (en), theme (system), app_lock (false), reminder_enabled (true), reminder_time (20:00), budget_alert_at (0.8), profile_name, last_period_home (month)

Indexes: `transactions(occurred_at)`, `transactions(category_id, occurred_at)`, `transactions(type, occurred_at)`.

8. Period and calculation rules

All periods use the phone's local time zone and are half-open ranges `[start, end)`, so a record at exactly midnight belongs to the new day.

Period	Start	End (exclusive)	Prev / next step
Today	00:00 today	00:00 tomorrow	1 day

Period	Start	End (exclusive)	Prev / next step
Week	00:00 on the week-start day (setting, default Monday)	Start + 7 days	7 days
Month	00:00 on the 1st	00:00 on the 1st of next month	1 month
Year	00:00 on 1 January	00:00 on 1 January next year	1 year
Custom	00:00 on the chosen first day	00:00 after the chosen last day	None

Calculations

- Income total = sum of `amount` where type = income in the range; expense total likewise.
- Net balance = income − expenses (shown with − when negative).
- Saved % = net ÷ income × 100, rounded; if income is 0 or net < 0, show *Overspent Rs X* instead.
- Budget % used = spent ÷ limit × 100; bar capped at 100%; state: < 80% normal, ⩾ 80% warning, = 100% fully used, > 100% over by (spent − limit).
- Safe to spend per day = (overall limit − spent this month) ÷ days left in the month including today; 0 if over budget.
- Home budget card per period: Today = daily limit (monthly limit ÷ days in month), Week = monthly limit ÷ days in month × 7, Month = monthly limit, Year = monthly limit × 12.
- Records day groups: label *Today*, *Yesterday*, else `Wed, 22 Oct`; group total = that day's income − expenses.

Money format: `Rs 12,345` in lists and totals (no decimals unless the amount has cents); short form `Rs 112k` / `Rs 1.91M` in report tiles. Income shows `+Rs`, expenses `−Rs`.

Recurring records: on each app start (and daily via a background check), every active rule whose `next_run_at` ⩽ now creates its record(s) and moves `next_run_at` forward one month; if the day doesn't exist (31st), use the month's last day.

9. Reports and charts

Three charts, all built with fl_chart, all recalculated live from the database for the selected period.

Chart	Type	Week	Month	Year
Income vs expenses	Grouped bars, income blue #1D5FCC, expenses orange #F08A4B	7 days, Mon–Sun	Last 6 months ending this month	12 months, Jan–Dec
Where the money went	Donut (pie with centre hole) + legend with % + breakdown list	Expenses this week	Expenses this month	Expenses this year
Spending pace	Two lines: cumulative spend this month (solid orange) vs last month (dashed grey)	Hidden	Shown	Hidden

Chart rules

- Y axis starts at 0 with 3 gridlines (0, half, max); max rounded up to a clean value (e.g. 220k).
- The current day / month label is bold; future days or months are greyed and draw no bars.
- Donut shows the top 5 categories; the rest merge into *Other*. Slices run largest first, clockwise from 12 o'clock, with a 2 px gap.
- Donut colours, in order: #0F1B2D, #1D5FCC, #7FB2F7, #B4471A, #F2A272, #A7B0BC (they differ in lightness, not only hue).
- Centre of donut shows total spent in short form.
- Tapping a bar or slice shows a tooltip with the exact amount; tapping a breakdown row opens Records filtered to that category and period.
- Spending pace subtitle states the difference at the same day of last month, e.g. *Rs 4,100 less than this point in September*.
- Empty period: show the chart frame with *No records yet* instead of zero-height bars.

10. Non-functional requirements

Area	Requirement
Platforms	Android 8.0 (API 26)+ first; iOS 14+ from the same codebase

Area	Requirement
Offline	Every feature works with no internet; no network calls in v1
Performance	Cold start to Home < 2 s; period switch and filters update < 200 ms with 10,000 records; charts render < 300 ms
Data safety	Writes are transactional; no data loss on app kill; backup file restores to an identical state
Privacy	Data stays on the device; no analytics or tracking in v1; receipts stored in app-private storage
Security	App lock uses the OS biometric prompt with PIN fallback; lock again after 1 minute in background
Accessibility	44 px touch targets, 4.5:1 text contrast, screen-reader labels, supports system font scaling to 130%
Localisation	All strings in ARB files (English first; Sinhala and Tamil ready); dates and numbers via `intl`
Reliability	No crashes on empty states, very large amounts (up to Rs 999,999,999) or deleted categories
Maintainability	Feature-based folders, repositories between UI and database, unit tests for all calculations
App size	Release APK under 25 MB

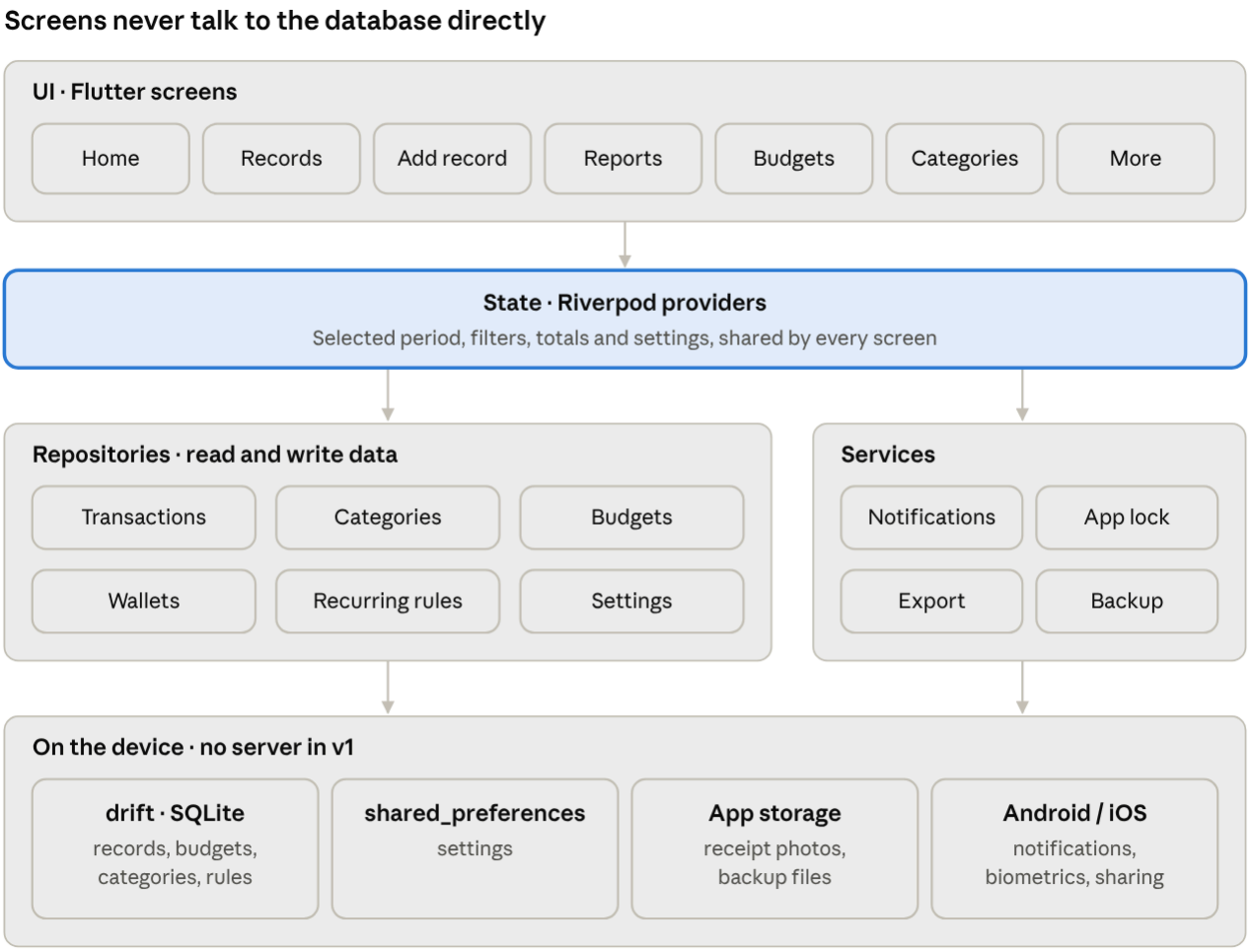
11. Tech stack and architecture

Flutter with a local SQLite database (drift) and Riverpod state management; no backend in v1.

Need	Package	Purpose
Framework	Flutter (Dart, stable channel)	One codebase for Android and iOS
Database	drift + drift_flutter (SQLite)	Typed tables and queries; streams that update screens live
State	flutter_riverpod	Selected period, filters, totals shared across screens

Need	Package	Purpose
Navigation	go_router	Bottom-bar shell + Add, Budgets, Categories routes
Charts	fl_chart	Bar, pie (donut) and line charts
Formatting	intl	Rs amounts, dates, week ranges, month names
Fonts	google_fonts (fonts bundled as assets)	Sora and Manrope, offline
Settings	shared_preferences	Key–value settings
Reminders	flutter_local_notifications	Daily reminder, budget alerts
App lock	local_auth	Fingerprint / face / device PIN
Receipts	image_picker + path_provider	Capture and store photos
Export	csv, pdf, share_plus	CSV and PDF export, share sheet
Code generation (dev)	drift_dev, build_runner	Generates database code
Testing (dev)	flutter_test, mocktail	Unit and widget tests

Layers: UI (screens and widgets) → Riverpod providers (state, filters, derived totals) → repositories (TransactionRepo, CategoryRepo, BudgetRepo, WalletRepo, RecurringRepo, SettingsRepo) → drift database and shared_preferences on the device. Services beside the repositories handle notifications, app lock, export and backup.



app architecture · 4 layers

Screens read and change data only through providers and repositories, so the database can later sync with Supabase without touching the UI.

Later (v2): add Supabase for login and cloud sync on top of the local database, keeping the app offline-first.

12. Local setup and project structure

The app runs on a local Android emulator, a USB-connected phone, or the desktop (Windows / macOS) for quick testing.

Prerequisites: Flutter SDK (stable), Android Studio with Android SDK and an emulator, VS Code or Android Studio as editor, Git. Run `flutter doctor` until it shows no errors.

Create and run

```
flutter create expense_manager
cd expense_manager
flutter pub add drift drift_flutter flutter_riverpod go_router fl_chart intl
google_fonts shared_preferences flutter_local_notifications local_auth
image_picker path_provider csv pdf share_plus
flutter pub add --dev drift_dev build_runner mocktail
dart run build_runner build --delete-conflicting-outputs
flutter run
```

- `flutter run -d windows` (or `-d macos`) for desktop testing; skip `-d chrome`, as drift needs extra web setup.
- Release build: `flutter build apk --release` (Android) or `flutter build ipa` (iOS, needs a Mac).
- Keep `expense-manager-prototype.html` in the project root as the design reference.

Folder structure

```
lib/
  main.dart                app start, ProviderScope, seed defaults
  app/                     router.dart, theme.dart (tokens), app_shell.dart
(bottom bar)
  data/
    database.dart          drift tables + migrations
    repositories/          transaction, category, budget, wallet, recurring,
settings
  domain/                  period.dart (ranges), money.dart (format),
calculations.dart
  features/
    home/  records/  add_record/  reports/  budgets/  categories/  settings/
  services/                notifications, app_lock, export, backup,
recurring_runner
  widgets/                 cat_icon, period_switch, balance_card,
record_tile, charts/
  l10n/                    app_en.arb
assets/fonts/              Sora, Manrope
test/                      unit tests (period, calculations), widget tests
```

13. Milestones and delivery plan

Six milestones, each ending with a build that runs on a phone; durations are estimates for one developer working with Claude Code.

1. **Foundation (about 1 week)** — project setup, theme and tokens, bottom-bar navigation, drift database with all tables, default category seeding, settings storage.

2. **Record expenses and income (about 1 week)** — Add / edit screen, wallets, validation, Undo, edit and delete (FR-06 to FR-15).
3. **Home and Records (about 1 week)** — period logic and arrows, balance card, recent list, Records filters, search, day grouping, custom range (FR-01 to FR-05, FR-16 to FR-21).
4. **Reports (about 1 week)** — summary tiles, bar, donut and line charts, breakdown list (FR-22 to FR-28).
5. **Budgets, categories, recurring (about 1 week)** — budgets and alerts, category management, recurring rules (FR-29 to FR-37, FR-45).
6. **Settings, security, data and release (about 1 week)** — app lock, reminders, theme, export, backup and restore, tests, icon and splash, release APK (FR-38 to FR-48).

14. Acceptance criteria and testing

v1 is accepted when every *Must* requirement passes these checks on a real Android phone.

- [ ] Adding an expense of Rs 2,250 in Groceries updates Home, Records, Reports and the Groceries budget instantly and identically.
- [ ] A record at 23:59 on Sunday appears in that week; one at 00:00 on Monday appears in the next week (week start Monday).
- [ ] Changing week start to Sunday changes every week total accordingly.
- [ ] Switching Today / Week / Month / Year on Home and Records gives the same totals for the same period.
- [ ] Amount validation rejects empty, 0, negative and more than 2 decimals.
- [ ] Editing a record's type from expense to income moves it between totals and requires an income category.
- [ ] Deleting a category with records asks where to move them; no orphan records remain.
- [ ] A budget at 80% shows the warning state and sends one notification (if enabled); over 100% shows *Over by Rs X*.
- [ ] Monthly recurring rent on the 31st creates a record on 30 November.
- [ ] Donut percentages add up to 100% and match the breakdown list.
- [ ] Export CSV opens in Excel / Google Sheets with correct dates and amounts; backup then restore on a fresh install gives identical data.
- [ ] App lock appears on open and after 1 minute in the background.
- [ ] Airplane mode: every feature still works.
- [ ] 10,000 sample records: period switch under 200 ms, no dropped frames when scrolling Records.

Automated tests: unit tests for period ranges, money formatting and all calculations in section 8; repository tests on an in-memory drift database; widget tests for Add record and the period switch.

15. Future enhancements and open questions

Future (v2 and later)

- Login and cloud sync with Supabase; use on several devices.
- Sinhala and Tamil translations.
- Shared household budgets.
- Bank SMS reading to suggest records (Android).
- Receipt OCR to fill the amount from a photo.
- Home-screen widget for quick add.
- Savings goals.

Open questions

- [ ] App name and icon — is "Expense Manager" the final name?
- [ ] Android only for the first release, or Android and iOS together?
- [ ] Should amounts show cents (Rs 2,250.50) or whole rupees only?
- [ ] Free app, or a paid / pro version later (e.g. cloud backup as a paid feature)?
- [ ] Backup target for v1: a local file only, or Google Drive as well?